

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 4

Total Marks:40

Annexure A

ARTICLE REVIEW

Code of Conduct:

*I **MADHU BAVANA.B (192524103)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MADHU BAVANA.B

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Article	20%	Demonstrates clear and in-depth understanding of the article's purpose, concepts, and arguments	Shows good understanding with minor gaps	Partial understanding; misses key ideas	Lacks understanding of the article
Summary	20%	Provides concise and accurate summary highlighting all key points	Covers most key points with minor omissions	Incomplete or partially accurate summary	Poor or incorrect summary
Critical Analysis	25%	Provides insightful critique with strong reasoning and evaluation	Provides reasonable analysis with some justification	Superficial analysis; limited evaluation	No meaningful analysis
Use of Evidence	15%	Effectively uses examples, data, or references to support review	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Organization	20%	Well-structured, clear, and coherent writing with proper language and flow	Generally clear with minor issues	Poor organization; lacks clarity	Disorganized and difficult to understand

Task 1: Article Summary

(a) Provide the full citation of the article (Author(s), Title, Journal/Conference, Year, DOI). State the domain/application area and the specific ML problem addressed.

(b) Summarise the key objective of the article in your own words (150–200 words). Clearly state the research gap the authors identified and how they proposed to fill it.

(a) Full Citation and AI Domain

- **Authors:** Óscar Pérez-Gil, Rafael Barea, Elena López-Guillén, Luis M. Bergasa, Carlos Gómez-Huélamo, Rodrigo Gutiérrez, Alejandro Díaz-Díaz¹
- **Title:** Deep reinforcement learning based control for Autonomous Vehicles in CARLA¹
- **Journal:** *Multimedia Tools and Applications*²
- **Year:** 2022 (Published online: 13 January 2022)²
- **DOI:** <https://doi.org/10.1007/s11042-021-11437-3>¹
- **Domain / Application Area:** Autonomous Vehicle (AV) control and navigation in dynamic urban environments¹more_horiz.
- **Specific ML/AI Problem:** Designing and training an AI-based **control layer** of an autonomous vehicle to solve **Markov Decision Processes (MDPs)**⁵⁶. The model must learn how to generate continuous, precise steering and speed/throttle commands to follow a predetermined route efficiently without collisions or lane departures⁴⁷.

(b) Summary (185 words)

The main objective of this research is to develop a **Deep Reinforcement Learning (DRL)** system integrated into the vehicle's control layer to follow global routes safely and efficiently¹⁴. The authors identify several critical research gaps: **classic controllers are highly environment-dependent**, requiring complex, non-trivial hyperparameter fine-tuning for new scenarios⁸. **Imitation learning systems**, while simpler, are restricted by training data and struggle to generalise to unseen, hazardous situations⁸. Furthermore, using discrete DRL algorithms like **Deep Q-Networks (DQN)** for autonomous vehicle control is sub-optimal and poorly studied because driving naturally requires continuous actions across an infinite spectrum of steering and throttle combinations⁴⁹.

To address these gaps, the authors implement and compare a discrete **DQN** with a continuous **Deep Deterministic Policy Gradient (DDPG)** controller¹⁴. They design four distinct agent

architectures—**Flatten-Image, Carla-Waypoints, CNN, and Pre-CNN**—to process different combinations of visual features and geometric waypoints¹⁰more_horiz. Through training and validation inside the hyper-realistic, open-source **CARLA simulator**, they demonstrate that continuous DDPG agents dramatically reduce training duration and achieve human-like, stable path-tracking¹more_horiz.

Task 2: Methodology Analysis

(a) Explain the ML technique(s) used in the article (e.g., Decision Tree variant, RL algorithm, Statistical model). Describe the dataset used—size, features, source, and how it was prepared.

(b) Map the technique to CO 4 topics (Inductive Learning / Decision Trees / Statistical Learning / RL). Identify one methodological strength and one limitation in the authors' approach.

(a) ML/RL Techniques and Simulation Setup

The study utilizes two primary DRL algorithms to solve MDPs:

1. **Deep Q-Network (DQN):** A discrete value-based method that approximates Q-values using a neural network, converting the continuous driving problem into **27 discrete classes** (a combination of 9 uniform steering positions in $[-1, 1]$ and 3 throttle positions in $[0, 0.5, 1]$)¹⁶¹⁷.
2. **Deep Deterministic Policy Gradient (DDPG):** A continuous Actor-Critic policy gradient method that concurrently learns a continuous Q-function (the Critic) and a policy (the Actor)¹⁸¹⁹. This maps neural network outputs directly to continuous steering $[-1, 1]$ and throttle 1 commands²⁰.

Both algorithms employ **experience replay buffers** to store past transitions, which stabilizes the training process and boosts data efficiency²¹.

Simulation Environment and State Space:

- **Environment: CARLA Simulator (version based on Unreal Engine 4)** with the OpenDrive standard defining the road network²²²³. Training takes place over dynamic routes generated by an **A* global planner** on random map points²⁴.
- **State Space Formulation** ($S = ([D], \theta_t, d_t)$): Driving features are extracted from CARLA at each step, consisting of vehicle speed (v_t), lateral distance to the

lane center (d_t), and heading angle relative to the lane direction (θ_t)²⁵²⁶. The input data D varies by agent:

- **DRL-Flatten-Image:** Black-and-white segmented frontal camera images scaled from 640×480 pixels down to 11×11 , flattened into a **121-component vector**¹⁰.
- **DRL-Carla-Waypoints: 15 localized waypoints** (relative to the ego-vehicle using translation and rotation transformation)²⁷²⁸. Only the **lateral x -coordinate** of each waypoint is used to reduce dimensionality¹².
- **DRL-CNN:** Raw **RGB images** (640×480) showing the highlighted drivable road²⁹. Visual features are processed online via a CNN with **three convolutional layers** (64 filters of sizes 7×7 , 5×5 , and 3×3 with ReLU activation and average pooling)²⁹.
- **DRL-Pre-CNN:** Utilizes the same CNN architecture, but **pre-trained offline** using a database of CARLA images to predict the local waypoints first¹³. The predicted waypoints are then fed into the DRL control networks¹³³⁰.
- **Actions:** Continuous or discrete commands for steering and throttle¹⁷²⁰. **Brake action is omitted** because the simulated environment contains no obstacles, meaning regenerative braking is sufficient to stop the car¹⁷.
- **Reward Function:** Penalizes collisions, lane departures, and lane changes with a value of -200 ¹⁷. It rewards reaching the goal with $+100$ ²⁰. Correct lane-keeping rewards speed along the heading while penalizing deviation: $R = \sum_t |v_t \cos \theta_t| - |v_t \sin \theta_t| - |d_t|$ ¹⁷

(b) CO 4 Mapping, Methodological Strengths, and Limitations

- **CO 4 Mapping: Reinforcement Learning (Model-Free, Continuous Actor-Critic vs. Discrete Value-Based Iteration)**^{1more_horiz}.
- **Methodological Strength: Pre-training the feature extractor offline (DRL-Pre-CNN).** By training the CNN offline to predict intermediate geometric waypoints, the authors decouple high-dimensional visual representation learning from policy training¹³. This effectively addresses the curse of dimensionality, allowing the DRL algorithm to learn lateral controls stably using a lightweight state representation¹³³².

- Methodological Limitation: Lack of a continuous braking mechanism in the action space.** Omitting the brake pedal and assuming a static, obstacle-free environment represents a major safety and behavioral limitation¹⁷. If applied to complex, dynamic urban scenarios with other traffic participants, the policy would be unable to stop the vehicle to prevent a collision¹⁷³³.

Task 3: Results & Critical Evaluation

- (a)** Report the key results (accuracy, F1-Score, AUC-ROC, reward convergence, etc.) as presented by the authors. Create a comparative table if multiple models are benchmarked.
- (b)** Critically evaluate the results: Are the reported metrics realistic? Are there potential biases (dataset, evaluation protocol)? What additional experiments would strengthen the findings?
- (c)** Discuss real-world applicability: Where can this technique be deployed? What are the challenges in scaling the solution to industry (data availability, compute, ethical issues)?

(a) Key Quantitative Results & Metric Comparisons

DDPG agents converged exponentially faster than DQN agents¹⁴. While DQN required at least **20,000 episodes** to yield a stable policy, DDPG reached its best performance within only **50 to 150 episodes** (representing a massive training time reduction)¹⁴³⁴.

Table 1: Training Performance Metrics³⁴

Method	Model	Total Training Episodes	Best Performance Episode
DQN ²⁷	DQN-Flatten-Image	20,000	16,500
DQN ²⁷	DQN-Carla-Waypoints	20,000	8,300
DQN ²⁷	DQN-CNN	120,000	108,600
DQN ²⁷	DQN-Pre-CNN	20,000	13,200
DDPG	DDPG-Flatten-Image	500	50
DDPG	DDPG-Carla-Waypoints	500	150
DDPG	DDPG-CNN	60,000	45,950
DDPG	DDPG-Pre-CNN	500	150

Table 2: Validation Metrics on Selected Town01 Route (180 meters, 20 iterations)³⁴³⁵

Controller Model	Root Mean Squared Error (RMSE)	Maximum Lateral Error	Average Time to Goal
LQR (Classic Control) ²¹	0.06 m	0.74 m	17.4 s
Manual Control ³⁶	0.40 m	1.80 m	22.7 s
DQN-Flatten-Image ²⁷	0.64 m	3.15 m	27.3 s
DQN-Carla-Waypoints ²⁷	0.21 m	1.32 m	29.3 s
DQN-CNN ²⁷	0.83 m	2.15 m	33.3 s
DQN-Pre-CNN ²⁷	0.33 m	1.72 m	28.2 s
DDPG-Flatten-Image	0.15 m	1.43 m	19.9 s
DDPG-Carla-Waypoints	0.13 m	1.50 m	20.6 s
DDPG-CNN	0.75 m	2.55 m	34.2 s
DDPG-Pre-CNN	0.10 m	1.41 m	23.8 s

Table 3: Robustness Validation for DDPG Agents (Mean of 20 novel routes, 180–700 meters)³⁷³⁸

Controller Model	Root Mean Squared Error (RMSE)	Maximum Lateral Error	Average Time to Goal
LQR (Classic Control) ²¹	0.095 m	1.305 m	65.60 s
DDPG-Flatten-Image	0.134 m	1.522 m	63.97 s
DDPG-Carla-Waypoints	0.100 m	1.460 m	62.25 s
DDPG-CNN	0.670 m	2.780 m	125.43 s
DDPG-Pre-CNN	0.115 m	1.512 m	65.12 s

(b) Critical Evaluation

- **Realism of Results:** The results are highly realistic within simulation constraints¹³⁹. The trajectories closely match the physical behavior of a classical **Linear-Quadratic Regulator (LQR)** controller, showing lateral trajectory errors that vary by only **4 to 7 centimeters**—an distance that is practically irrelevant in relation to lane width³⁵.
- **Dataset & Evaluation Biases:** The evaluation is restricted entirely to **Town01 in the CARLA simulator** under static weather and clear lighting conditions³⁴³⁹. There is a lack of dynamic diversity (no other vehicles, traffic signals, or pedestrians)¹⁷³³. Furthermore, comparing the Carla-Waypoints model directly to other agents is slightly biased because it leverages **ground-truth, noise-free local waypoints** supplied directly by the CARLA PythonAPI, which bypassed the visual perception pipeline¹¹.
- **Additional Experiments Needed:**
 1. **Dynamic Obstacle Scenarios:** Introducing pedestrians and other vehicles to test collision avoidance and evaluate actual braking policies¹⁷³³.
 2. **Cross-Map Evaluation:** Validating the trained models on entirely different CARLA layouts (e.g., Town02 or Town03) to test spatial generalization⁴⁰⁴¹.
 3. **Noisy Waypoint Injection:** Adding synthetic sensor noise (Gaussian distribution) to the local waypoints to evaluate control robustness against perception errors⁴².

(c) Industrial Applicability & Challenges

- **Safety and Stochasticity:** Conventional control architectures utilize deterministic systems with stable performance guarantees⁴². AI-based DRL control is stochastic, meaning its learning can be poor depending on hyperparameters, network architecture, and data quality⁴². Running early trial-and-error training directly on physical vehicles is impossible as random early actions would destroy equipment or threaten lives⁴³.
- **Computation & Scalability:** Standard online visual training (such as DDPG-CNN) processes over **300,000 pixels per frame**, leading to severe learning bottlenecks³². This requires expensive high-end GPUs (e.g., RTX 2080 Ti) and large memory resources, which are difficult to scale on low-power, embedded vehicular systems³²⁴⁴.
- **Sim-to-Real Transfer (The Reality Gap):** Transitioning these models to a physical vehicle requires generating highly detailed, centimeter-level maps of the real

environment to train the agent in CARLA beforehand³³. Additionally, the software architecture must support **ROS (Robot Operating System) bridge integration** to safely manage latency and real-world hardware control loops³³.

Task 4: Reflection & Learning Outcome

(a) State three key insights you gained from reading the article that deepen your understanding of CO 4 topics. Connect each insight to a specific concept from the course syllabus.

(b) If you were to extend this research, propose one novel experiment or enhancement (different dataset / improved algorithm / new application domain). Justify its feasibility and expected impact.

(a) Key Insights & Course Concept Connections

1. Continuous Policy vs. Value Discretization:

- *Insight:* Continuous action spaces (DDPG) map perfectly to physical vehicular controls⁴²⁰. Jerky, uniform discretization of steering and throttle (as seen in DQN's 27 classes) complicates the driving task, yielding high oscillations, slow convergence, and sub-optimal lateral error⁹.
- *Connection:* **Reinforcement Learning (Continuous Action Control & Actor-Critic Architectures)**¹⁹³¹. Shows how model-free approaches benefit from policy parameterization over tabular value iteration in complex, multi-dimensional control loops³¹.

2. Structural Decoupling via Representation Learning:

- *Insight:* Attempting to optimize visual perception and control policies simultaneously online (end-to-end CNN agent) fails to scale³². Predicting low-dimensional spatial landmarks (waypoints) via offline supervised CNNs first allows the DRL agent to converge instantly¹³.
- *Connection:* **Statistical Learning / Feature Engineering & Representation Learning**¹³⁴². Emphasizes the importance of using inductive biases (local waypoints) to drastically simplify the state vector representation before feeding it into downstream models²⁵³².

3. Sample Autocorrelation Mitigation:

- *Insight:* DRL algorithms assume independent training samples²¹. However, in autonomous driving, successive frames are highly sequential and correlated

because state s_{t+1} is a direct consequence of action a_t ²¹. Experience replay buffers are necessary to break this temporal correlation²¹.

- *Connection:* **Reinforcement Learning (Markov Decision Processes & Temporal Difference Learning)**⁶⁴⁵. Illustrates how replay buffers ensure data distribution stability, allowing single transitions to be evaluated under multiple policies²¹.

(b) Proposed Novel Experiment: "TD3-Hybrid Dynamic Navigation"

- **Concept:** Replace the baseline DDPG algorithm with **Twin Delayed DDPG (TD3)**, integrating a continuous three-dimensional action space: **Steering, Throttle, and Brake**³¹⁴⁶.
- **Technical Enhancement:** TD3 resolves the classic overestimation bias of DDPG by utilizing **clipped double Q-learning** and **delayed policy updates**³¹. The agent's state vector will be extended to include distance-to-obstacle data from a simulated 3D Lidar point cloud².
- **Feasibility: High.** The current simulation framework uses CARLA's PythonAPI²². CARLA features built-in collision sensors and Scenario Runner to spawn dynamic obstacles (pedestrians and target cars)²²⁴⁷. The existing DDPG Actor-Critic neural network structure can be easily modified to output a third continuous dimension (Brake) in `1more_horiz`.
- **Expected Impact:**
 - **Overestimation bias mitigation** in TD3 will result in smoother control trajectories and higher training stability³¹.
 - Evaluating the vehicle's capacity to decelerate dynamically in response to other traffic participants will bridge the critical safety gap, preparing the algorithm for actual real-world, ROS-integrated physical driving applications